Note on the Model Implementation

1. Architecture of the `swaptionPricingG2PP` Function

At the current stage of the implementation, the main difficulty is no longer the derivation of the pricing formula, but rather the correct numerical and structural organization of the code. The objective is therefore to design the `swaptionPricingG2PP` function as a clean orchestration layer built on top of smaller deterministic and analytic helper functions.

The central principle is the following:

> *The main swaption pricing function should not itself be the theorem. It should orchestrate a sequence of precomputed deterministic objects, a Gaussian parameter block, an inner nonlinear solve, and an outer one-dimensional integration.*

This principle is especially important in the present G2++ setting, where the final European swaption formula contains both:

1. an inner scalar nonlinear solve for the critical value $\bar{y}(x)$,
2. an outer one-dimensional Gaussian integral in the variable x.

1.1. Three-layer architecture

The recommended implementation is naturally divided into three layers.

Layer A: low-level scalar/vector kernels

These functions should accept only the precise numerical objects they need. They are direct transcriptions of the theorem and should not recompute model quantities internally.

Typical examples are:

$$h_1(x), \qquad h_{2,i}(x), \qquad \lambda_i(x), \qquad \kappa_i(x),$$

as well as the objective function defining $\bar{y}(x)$ and its derivative.

The reason for this choice is straightforward:

- these objects are called repeatedly inside the quadrature loop,
- they should remain mathematically transparent,
- they should be easy to unit-test independently,
- hidden recomputations must be avoided.

In particular, the function computing $h_1(x)$ should not itself solve for $\bar{y}(x)$, nor should it query the model object to rebuild $\mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy}$. Instead, it should take those quantities as explicit inputs.

Layer B: model-dependent swaption builders

These functions may use the full model object `obj`, because they assemble deterministic quantities that depend on the current G2++ specification, the term structure, the volatility buckets, and the instrument conventions.

Typical examples are:

- the instrument block,
- the bond basket block,
- the forward Gaussian parameter block.

These functions should be called once per instrument, not once per quadrature node.

Layer C: orchestration functions

These functions coordinate the pricing logic:

- solving the critical value $\bar{y}(x)$,
- evaluating the integrand at a given node x,
- performing the final numerical quadrature.

At this level, one should avoid embedding algebraic details in the main function body. The role of the orchestration layer is simply to call the correct lower-level components in the correct order.

1.2. Deterministic objects to precompute outside the integrand

For one fixed swaption, all quantities that do not depend on the integration variable x should be precomputed once, before launching the outer quadrature.

These include:

$$P(0,T_0), \qquad \{c_i\}_{i=1}^n, \qquad \{A_i\}_{i=1}^n, \qquad \{B_{x,i}\}_{i=1}^n, \qquad \{B_{y,i}\}_{i=1}^n, \tag{1}$$

together with the Gaussian parameters

$$\mu_x, \qquad \mu_y, \qquad \sigma_x, \qquad \sigma_y, \qquad \rho_{xy}. \tag{2}$$

One should also precompute useful scalar combinations such as

$$\sigma_y\sqrt{1-\rho_{xy}^2}, \qquad \frac{1}{\sigma_x\sqrt{2\pi}}, \qquad \frac{1}{\sigma_x\sqrt{1-\rho_{xy}^2}}, \qquad \frac{1}{\sigma_y\sqrt{1-\rho_{xy}^2}}. \tag{3}$$

The numerical integrand should therefore not perform any model-level work beyond:

1. solving $\bar{y}(x)$,
2. evaluating $h_1(x)$,
3. evaluating $h_{2,i}(x)$,
4. evaluating $\lambda_i(x)$,
5. evaluating $\kappa_i(x)$,
6. assembling the final scalar integrand.

1.3. Separation of the critical-value solver

A key architectural point is that the nonlinear solve for $\bar{y}(x)$ must remain separate from the theorem objects that depend on it.

The natural sequence is

$$x \longrightarrow \bar{y}(x) \longrightarrow h_1(x) \longrightarrow h_{2,i}(x).$$

This implies in particular:

- the solver for $\bar{y}(x)$ is a dedicated numerical routine;
- $h_1(x)$ should use the already-computed value $\bar{y}(x)$;
- $h_2(x)$ should ideally use $h_1(x)$ plus the additional deterministic shift.

Thus, h_2 should not be treated as a second full theorem object. It is merely

$$h_{2,i}(x) = h_1(x) + B_y(T_0,T_i)\,\sigma_y\sqrt{1-\rho_{xy}^2}. \tag{4}$$

1.4. Numerical integration strategy in MATLAB

The outer integral is formally defined on the whole real line. In practice, the recommended strategy is to standardize the Gaussian variable:

$$x = \mu_x + \sigma_x z. \tag{5}$$

Under this transformation, the integral becomes one with respect to the standard normal density in z. The numerical integration may then be performed on a bounded interval

$$z \in [-L, L], \tag{6}$$

with a first practical choice such as

$$L = 6 \quad \text{or} \quad L = 7.$$

This approach is particularly attractive here because:

- the library already provides a bounded-interval Gaussian quadrature;
- the integrand is not merely a polynomial correction of a Gaussian kernel;
- each node evaluation requires the inner nonlinear solve for $\bar{y}(x)$, so implementation robustness is more important than formal elegance.

Consequently, a bounded quadrature after Gaussian standardization is an entirely reasonable first implementation, and often preferable to jumping immediately to Gauss–Hermite quadrature.

1.5. Warm-start strategy for the inner nonlinear solver

The most important practical numerical advice is to exploit the smooth dependence of $\bar{y}(x)$ on x.

Indeed, the quadrature nodes $x_1, x_2, \ldots$ are ordered and typically close to one another. Hence the critical value $\bar{y}(x)$ varies smoothly as a function of x. This suggests the following strategy:

1. solve $\bar{y}(x_1)$ using a generic initial guess;
2. use $\bar{y}(x_1)$ as the initial guess for $\bar{y}(x_2)$;
3. continue recursively along the ordered quadrature nodes.

This warm-start policy can significantly reduce the computational burden of the swaption pricing routine.

1.6. Recommended package structure

It is advantageous to conceptually group the deterministic quantities into a single package:

$$\mathcal{P} = \{P(0,T_0),\ \{c_i\}_{i=1}^n,\ \{A_i\}_{i=1}^n,\ \{B_{x,i}\}_{i=1}^n,\ \{B_{y,i}\}_{i=1}^n,\ \mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy}\}\,. \tag{7}$$

Then one may think of the numerical architecture as:

$$\texttt{solveYBar}(x, \mathcal{P}), \qquad \texttt{swaptionIntegrandX}(x, \mathcal{P}), \qquad \texttt{swaptionPricingG2PP}(\mathcal{P}).$$

Such a structure avoids the propagation of too many loose scalar arguments and keeps the pricing logic readable.

1.7. Immediate implementation roadmap

At the current stage, the next coding steps should follow the sequence:

1. finalize the Gaussian parameter block;
2. finalize the swaption package by combining:
   - the instrument block,
   - the bond basket block,
   - the Gaussian block;
3. define the low-level theorem kernels:
$$h_1, \quad h_2, \quad \lambda_i, \quad \kappa_i;$$
4. finalize the scalar objective function defining $\bar{y}(x)$ and its derivative;
5. finalize the nonlinear solver for $\bar{y}(x)$;
6. define the scalar integrand in the variable x;
7. finally, define the public pricing function `swaptionPricingG2PP` as an orchestration routine around bounded Gaussian quadrature.

2. Current architecture of the G2++ swaption pricing block

At the present stage, the deterministic layer of the G2++ swaption implementation is already largely in place. In particular, the forward-measure Gaussian block, the swaption instrument normalization, the bond basket construction, and the nonlinear solver for the critical value $\bar{y}(x)$ have all been separated into dedicated helper functions.

The implementation goal is therefore no longer to derive new formulas, but to organize the pricing routine in a numerically robust and structurally clear way.

The European swaption pricing routine should be viewed as the composition of four blocks:

1. the forward-measure Gaussian parameter block;
2. the instrument and bond basket block;
3. the nonlinear solver for $\bar{y}(x)$;
4. the one-dimensional integration over the x-variable.

2.1. Forward-measure Gaussian block

The first block builds the Gaussian law of the pair $(x(t), y(t))$ under the T-forward measure. In the current implementation, this block is organized through the following helper functions:

$$\texttt{muTForward}, \qquad \texttt{varTForward}, \qquad \texttt{covTForward}. \tag{8}$$

Their role is the following.

Drift-shift / mean block. The function `muTForward(obj,startDate,endDate,args)` returns the forward-measure drift correction associated with either the x-process or the y-process. Conceptually, it computes

$$-M_x^T(s,t) \qquad \text{or} \qquad -M_y^T(s,t), \tag{9}$$

depending on the argument

$$\texttt{args.process} \in \{\texttt{'x_process'}, \texttt{'y_process'}\}.$$

The additional argument `tMaturity` is used to distinguish between:

- the general forward-measure case (s, t, T),
- the swaption specialization $T = t$, where the end date is also the forward-measure maturity.

Variance block. The function `varTForward(obj,startDate,endDate,args)` returns the conditional variance of one OU factor over the interval $[s, t]$,

$$\mathrm{Var}^{Q^T}\big[x(t) \mid \mathcal{F}_s\big] \qquad \text{or} \qquad \mathrm{Var}^{Q^T}\big[y(t) \mid \mathcal{F}_s\big]. \tag{10}$$

By construction, this object depends on the interval $[s, t]$ only, and not on the forward maturity T.

Covariance block. The function `covTForward(obj,startDate,endDate)` returns the covariance

$$\mathrm{Cov}^{Q^T}\big[x(t), y(t) \mid \mathcal{F}_s\big]. \tag{11}$$

Again, this object depends on the interval $[s, t]$, but not on the forward maturity T.

This leads to the conceptual split:

$$\boxed{\text{means depend on } (s, t, T), \qquad \text{variances and covariance depend on } (s, t).} \tag{12}$$

For the swaption pricing problem, the specialization is

$$s = 0, \qquad t = T_0, \qquad T = T_0, \tag{13}$$

where T_0 is the swaption expiry.

2.2. Swaption instrument and bond basket block

The second block transforms the swaption contract specifications into the deterministic objects entering the affine pricing formula.
It is currently organized through the two helpers

$$\texttt{getSwaptionInstrument}, \qquad \texttt{getSwaptionBondBasket}. \tag{14}$$

Instrument block. The function `getSwaptionInstrument(obj,strike,currMaturity,ulSwapSteps,args)` standardizes the swaption contract conventions. Its role is to:

- convert dates from datenum format to year-fraction format when needed;
- normalize the underlying swap payment dates
$$T_1, \ldots, T_n;$$
- compute the accruals
$$\tau_i = T_i - T_{i-1};$$
- build the fixed-leg coefficients
$$c_i;$$
- store the payer/receiver convention and the notional.

Thus, this block returns a compact specification of the swaption instrument in a format suitable for the pricing formula.

Bond basket block. The function `getSwaptionBondBasket(obj,currMaturity,ulSwapSteps)` builds the affine bond basket associated with the underlying swap. Its outputs are the vectors

$$P(0, T_0), \qquad \{P(0, T_i)\}_{i=1}^n, \qquad \{A(T_0, T_i)\}_{i=1}^n, \qquad \{B_x(T_0, T_i)\}_{i=1}^n, \qquad \{B_y(T_0, T_i)\}_{i=1}^n. \tag{15}$$

This is the deterministic affine representation entering the Brigo–Mercurio theorem. More precisely, for each payment date T_i,

$$P(T_0, T_i) = A(T_0, T_i)\, \exp\big(-B_x(T_0, T_i)x(T_0) - B_y(T_0, T_i)y(T_0)\big). \tag{16}$$

Hence, the instrument block and the bond basket block together provide the deterministic coefficients of the swaption payoff viewed as a zero-coupon bond basket.

2.3. Nonlinear solver block

The third block solves the scalar nonlinear equation defining the critical value $\bar{y}(x)$ in the swaption pricing formula. The current helper functions are:

$$\texttt{getLambdaAndBy}, \qquad \texttt{getYBarFunc}, \qquad \texttt{yBarSolver}. \tag{17}$$

Lambda builder. The function `getLambdaAndBy` builds the vectors

$$\lambda_i(x) = c_i\, A(T_0, T_i)\, e^{-B_x(T_0,T_i)x}, \qquad B_{y,i} = B_y(T_0, T_i). \tag{18}$$

These are precisely the objects entering the scalar root problem.

Objective function. The function `getYBarFunc(obj,y,lambda,By)` returns the scalar objective

$$f(y) = 1 - \sum_{i=1}^{n} \lambda_i(x)\, e^{-B_{y,i}y}, \tag{19}$$

together with its derivative

$$f'(y) = \sum_{i=1}^{n} \lambda_i(x)\, B_{y,i}\, e^{-B_{y,i}y}. \tag{20}$$

Scalar solver. The function `yBarSolver` then solves

$$f(y) = 0 \tag{21}$$

using a two-stage strategy:

1. first, the internal library Newton solver `newtonVector`,
2. if needed, a fallback routine using MATLAB's native scalar root solver.

This organization is numerically natural because the map $y \mapsto f(y)$ is scalar, smooth, and monotone under standard swaption conditions.

2.4. One-dimensional integration block

The last block is the numerical integration over the x-variable in the Brigo–Mercurio one-dimensional representation. At a high level, the computational chain is:

$$x \longrightarrow \bar{y}(x) \longrightarrow h_1(x) \longrightarrow h_{2,i}(x),\ \lambda_i(x),\ \kappa_i(x) \longrightarrow \text{integrand}(x). \tag{22}$$

This implies a crucial implementation principle:

> *The theorem-level helper functions should not recompute model-level quantities. They should receive explicit scalar or vector inputs already assembled by the higher-level deterministic blocks.*

For example, the function computing $h_1(x)$ should not:

- solve for $\bar{y}(x)$,
- rebuild $\mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy}$,
- query the instrument specification again.

Instead, it should receive:

$$x, \qquad \bar{y}(x), \qquad \mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy},$$

and perform only the local scalar calculation.
Similarly, the function computing $h_2(x)$ should be a thin wrapper around

$$h_{2,i}(x) = h_1(x) + B_y(T_0, T_i)\, \sigma_y \sqrt{1 - \rho_{xy}^2}. \tag{23}$$

2.5. Precomputation principle

For one fixed swaption, all deterministic quantities that do not depend on the integration variable x should be precomputed once and for all before entering the outer integration loop.
These include:

$$P(0, T_0), \qquad \{c_i\}_{i=1}^n, \qquad \{A_i\}_{i=1}^n, \qquad \{B_{x,i}\}_{i=1}^n, \qquad \{B_{y,i}\}_{i=1}^n, \tag{24}$$

together with

$$\mu_x, \qquad \mu_y, \qquad \sigma_x, \qquad \sigma_y, \qquad \rho_{xy}. \tag{25}$$

Hence the outer integrand should only perform:

1. evaluation at the current node x,
2. computation of $\bar{y}(x)$,
3. evaluation of the theorem-level scalar/vector objects,
4. assembly of the scalar integrand.

This avoids hidden recomputation and keeps the pricing function readable.

2.6. Numerical perspective

The current numerical structure also suggests the following practical strategy for the final pricing routine.

First, one may standardize the Gaussian variable as

$$x = \mu_x + \sigma_x z, \tag{26}$$

so that the integration is performed with respect to the standard normal density.

Second, the integral may be truncated to a bounded interval

$$z \in [-L, L], \tag{27}$$

for instance with $L = 6$ or $L = 7$, and then evaluated by a bounded Gaussian quadrature available in the library.

Third, the sequence of nodes should be used to warm-start the nonlinear solver for $\bar{y}(x)$: once $\bar{y}(x_k)$ has been computed, it should serve as the initial seed for $\bar{y}(x_{k+1})$.

This is especially useful because the expensive part of the algorithm is not the Gaussian weight itself, but rather the repeated inner nonlinear solve.

2.7. Current architectural view

Given the current implementation state, the architecture of the swaption pricing block may therefore be summarized as

$$\boxed{\texttt{swaptionPricingG2PP} = \texttt{GaussianBlock} + \texttt{InstrumentBlock} + \texttt{BondBasketBlock} + \texttt{YBarSolver} + \texttt{OneDimensionalIntegra}} \tag{28}$$

The relevant helper functions currently in place are:

$$\begin{aligned} &\texttt{muTForward}, \quad \texttt{varTForward}, \quad \texttt{covTForward}, \\ &\texttt{getSwaptionInstrument}, \quad \texttt{getSwaptionBondBasket}, \\ &\texttt{getLambdaAndBy}, \quad \texttt{getYBarFunc}, \quad \texttt{yBarSolver}. \end{aligned} \tag{29}$$

This is already a coherent and promising structure. The next implementation step is therefore not to enlarge the mathematical scope, but rather to preserve this decomposition while finalizing the theorem-level integrand objects and the outer numerical quadrature.

2.8. Guiding MATLAB snippets

The following snippets are not meant to be complete implementations. Their role is to clarify the intended architecture, the separation of responsibilities between helper functions, and the role of a `swaptionPackage` object gathering all deterministic quantities attached to one swaption.

The central implementation principle is the following:

> *All model-dependent deterministic objects should be assembled once, outside the quadrature loop. The theorem-level helper functions should then receive only explicit scalar or vector inputs, and should not trigger hidden recomputations of model quantities.*

In particular, the one-dimensional integration block should not repeatedly rebuild:

$$\{c_i\}_{i=1}^n, \qquad \{A_i\}_{i=1}^n, \qquad \{B_{x,i}\}_{i=1}^n, \qquad \{B_{y,i}\}_{i=1}^n, \qquad \mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy}.$$

These quantities belong to the deterministic package of the swaption and should be computed only once.

Low-level scalar/vector kernel style

These functions should receive only the precise numerical objects they need. They should not query the model object in order to recompute already available deterministic data.

For example, the function computing $h_1(x)$ should take as input:

$$x, \qquad \bar{y}(x), \qquad \mu_x, \mu_y, \sigma_x, \sigma_y, \rho_{xy},$$

and nothing more.

```
function output = h1Function(obj, xVal, yBarVal, mu_x, mu_y, sigma_x, sigma_y, rho_xy)
    arguments
        obj (1,1) GaussianTwoFactors
        xVal (1,1) double
        yBarVal (1,1) double
        mu_x (1,1) double
        mu_y (1,1) double
        sigma_x (1,1) double
        sigma_y (1,1) double
        rho_xy (1,1) double
    end

    % purely local computation, no hidden recomputation
end
```

This same principle applies to:

- $h_{2,i}(x)$,
- $\lambda_i(x)$,
- $\kappa_i(x)$,
- the objective function defining $\bar{y}(x)$,
- the derivative of that objective.

The reason is twofold:

1. these objects are called repeatedly inside the numerical integration routine;
2. they correspond directly to theorem-level expressions and should remain transparent and testable.

Using $\bar{y}(x)$ outside of h_1

The critical value $\bar{y}(x)$ should be solved first. The result should then be passed explicitly to the theorem-level helper functions.

The intended chain is:

$$x \longrightarrow \bar{y}(x) \longrightarrow h_1(x) \longrightarrow h_{2,i}(x), \lambda_i(x), \kappa_i(x) \longrightarrow \text{integrand}(x).$$

Thus, the helper computing $h_1(x)$ should not itself contain a call to the nonlinear solver. Likewise, $h_2(x)$ should not rebuild $h_1(x)$ from scratch if the latter has already been computed.

```
function output = swaptionIntegrandX(obj, xVal, swaptionPackage, yInitialGuess)
    arguments
        obj (1,1) GaussianTwoFactors
        xVal (1,1) double
        swaptionPackage struct
        yInitialGuess (1,1) double
    end

    % 1) solve yBar(x)
    % 2) compute h1
    % 3) compute h2_i vector
    % 4) compute lambda_i vector
    % 5) compute kappa_i vector
    % 6) assemble scalar integrand
end
```

In this architecture, the nonlinear solver is a numerical device, not a theorem object. It should therefore remain outside the scalar formula helpers.

Model-level assembly block style

The Gaussian block is model-dependent. It is therefore natural for this type of helper to use the full model object. Its role is to assemble the Gaussian parameters entering the swaption theorem:

$$\mu_x, \qquad \mu_y, \qquad \sigma_x, \qquad \sigma_y, \qquad \rho_{xy}.$$

In the swaption specialization, these are computed at

$$s = 0, \qquad t = T_0, \qquad T = T_0,$$

where T_0 is the swaption expiry.

```
function output = getSwaptionGaussianBlock(obj, currMaturity)
    arguments
        obj (1,1) GaussianTwoFactors
        currMaturity (1,1) double
    end

    % build:
    %   mu_x, mu_y
    %   sigma_x, sigma_y
    %   rho_xy
    %
    % swaption specialization:
    %   startDate = 0
    %   endDate   = currMaturity
    %   tMaturity = currMaturity
end
```

More generally, the deterministic assembly functions are precisely the ones allowed to use:

- the model parameters,
- the volatility buckets,
- the curve objects,
- the market conventions,
- the instrument schedule.

These should be called once per instrument, not once per quadrature node.

The role of the `swaptionPackage`

A useful implementation device is to gather all deterministic quantities associated with one swaption into a single MATLAB structure, denoted here by `swaptionPackage`.

The purpose of this object is to avoid propagating a long list of scalar and vector arguments across all helper functions. Instead of passing

$$c_i, \quad A_i, \quad B_{x,i}, \quad B_{y,i}, \quad \mu_x, \quad \mu_y, \quad \sigma_x, \quad \sigma_y, \quad \rho_{xy}, \quad \omega, \quad N,$$

one passes a single container whose fields are read by the integrand and solver layers.

Conceptually, the `swaptionPackage` is the merge of three deterministic blocks:

1. the instrument block;
2. the bond basket block;
3. the Gaussian parameter block.

Thus one may think of it as

$$\texttt{swaptionPackage} = \texttt{InstrumentPackage} \cup \texttt{BondBasketPackage} \cup \texttt{GaussianPackage}. \tag{30}$$

A typical content would include:

$$\begin{aligned}
&\texttt{expiry}, \quad \texttt{paymentDates}, \quad \texttt{tenors}, \quad \texttt{coeffs}, \quad \texttt{direction}, \quad \texttt{omega}, \quad \texttt{notional}, \\
&\texttt{P0T0}, \quad \texttt{P0Ti}, \quad \texttt{A0Ti}, \quad \texttt{Bx0Ti}, \quad \texttt{By0Ti}, \\
&\texttt{mu_x}, \quad \texttt{mu_y}, \quad \texttt{sigma_x}, \quad \texttt{sigma_y}, \quad \texttt{rho_xy}.
\end{aligned} \tag{31}$$

One may then define a dedicated builder such as:

```
function output = getSwaptionPackage(obj, strike, currMaturity, ulSwapSteps, args)
    arguments
        obj (1,1) GaussianTwoFactors
        strike (1,1) double
        currMaturity (1,1) double
        ulSwapSteps (1,:) double
        args.direction (1,:) char = 'PAY'
        args.notional (1,1) double = 1e6
    end

    % 1) instrument = obj.getSwaptionInstrument(...)
    % 2) basket     = obj.getSwaptionBondBasket(...)
    % 3) gaussian   = obj.getSwaptionGaussianBlock(...)
    % 4) aggregate all fields into one output struct
end
```

This builder is especially valuable because it centralizes the deterministic layer. Once it exists, the main pricing function can operate almost entirely through that package.

The intended usage is then:

```
swaptionPackage = obj.getSwaptionPackage(strike, currMaturity, ulSwapSteps, ...);

yBarVal = obj.yBarSolver(xVal, swaptionPackage, ...);
integrandVal = obj.swaptionIntegrandX(xVal, swaptionPackage, yGuess);
```

This approach is not only cleaner, but also makes later testing and debugging easier:

- one may inspect the package directly in MATLAB;
- one may verify that all affine and Gaussian objects are coherent before entering the solver;
- one may isolate numerical issues from deterministic-construction issues.

Outer quadrature wrapper style

The public pricing function should remain short. Its role is to coordinate the blocks rather than to contain all the formula details inline.

In particular, the outer quadrature wrapper should:

1. normalize the swaption specification,
2. build the deterministic package,
3. standardize the Gaussian integration variable,
4. call the scalar integrand repeatedly,
5. aggregate the numerical quadrature output.

```
function output = swaptionPricingG2PP(obj, strike, currMaturity, ulSwapSteps, args)
    arguments
        obj (1,1) GaussianTwoFactors
        strike (1,1) double
        currMaturity (1,1) double
        ulSwapSteps (1,:) double
        args.direction (1,:) char = 'PAY'
        args.notional (1,1) double = 1e6
    end

    % 1) build swaptionPackage
    % 2) standardize x = mu_x + sigma_x * z
    % 3) choose bounded interval [-L, L]
    % 4) evaluate the scalar integrand on quadrature nodes
    % 5) multiply by the deterministic prefactor
end
```

The main routine should therefore look more like an orchestration script than a theorem derivation.

Warm-start logic for the inner solver

Because $\bar{y}(x)$ varies smoothly with x, the previous root should be recycled as the initial guess for the next quadrature node.

This point is important because the dominant cost of the numerical routine is not the Gaussian weight itself, but the repeated inner nonlinear solve.

```
yGuess = 0.0;
for k = 1:nNodes
    xVal = xNodes(k);

    % solve yBar(xVal) with previous root as new initial guess
    yBarVal = ...;
    yGuess = yBarVal;

    % evaluate integrand
    integrandVals(k) = ...;
end
```

This warm-start policy is especially natural when the quadrature nodes are ordered. In that case, successive values of $\bar{y}(x)$ are typically close to one another, and the solver can converge much faster.

Final practical guideline

The intended separation of responsibilities may finally be summarized as follows:

- **Low-level theorem kernels:** accept explicit scalar/vector inputs only.
- **Model-dependent builders:** use `obj` to assemble deterministic objects.
- **Swaption package builder:** gather all deterministic objects once into a single structure.
- **Solver and integrand layer:** use the package, but avoid rebuilding it.
- **Public pricing function:** remain short and only orchestrate the previous blocks.

This is the implementation philosophy that best matches both the current G2++ structure and the good practices already visible in the Hull–White code base.

2.9. Conclusion

The implementation principle may be summarized as follows:

> *All model-dependent deterministic objects should be assembled once, outside the quadrature loop; all theorem-level kernels should operate only on explicit scalar or vector inputs; the main pricing function should remain a short orchestration layer around the instrument block, the bond basket block, the Gaussian block, the inner nonlinear solver, and the bounded Gaussian quadrature.*

This architecture is both numerically robust and consistent with the implementation philosophy already present in the Hull–White code base.